Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase IV · Escala, sistemas y arquitecturas modernas

Módulo 17

Hardware y sistemas distribuidos

Aceleradores, memoria, precisión y paralelismo en el entrenamiento de modelos de lenguaje

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 17: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
F, Q	número de operaciones en coma flotante (FLOP) y bytes transferidos de un núcleo de cálculo
P_{pico}, B_{mem}	rendimiento máximo (FLOP/s) y ancho de banda de memoria (bytes/s) del dispositivo
$I = F/Q$, $I^* = P_{\text{pico}}/B_{\text{mem}}$	intensidad aritmética e intensidad crítica (FLOP/byte)
s	bytes por número (2 en bf16 y fp16, 4 en fp32)
M	capacidad de la memoria rápida (SRAM), en elementos
n, d	en el tema 3, longitud de la secuencia y dimensión de cada cabeza de atención
N, D, $C \approx 6ND$	parámetros del modelo, tokens de entrenamiento y cómputo total
p, $\varepsilon = 2^{-p}$, $u = \varepsilon/2$	bits de mantisa, épsilon de la máquina y unidad de redondeo
$\mathrm{fl}(x)$, $\mathrm{ulp}(x)$	redondeo al número representable más cercano y separación entre representables junto a x
n_{d}	número de dispositivos (GPU)
S, W	tamaño de un tensor comunicado (bytes) y ancho de banda de enlace (bytes/s)
p_{e}, m	etapas del pipeline y número de microlotes
Ψ	número de parámetros en el análisis de ZeRO
MFU, PUE	utilización de FLOP del modelo y eficiencia energética del centro de datos
k_{B}, T	constante de Boltzmann y temperatura absoluta

Índice

Sobre este resumen **ii**

1 Cómputo matricial y aceleradores **1**
1.1 Contar operaciones . . . 1
1.2 La regla de seis operaciones por parámetro y token . . . 1
1.3 Arquitectura de una GPU . . . 2
1.4 Matrices sistólicas . . . 2

2 Jerarquía de memoria y modelo roofline **4**
2.1 La jerarquía de memoria . . . 4
2.2 Intensidad aritmética y modelo roofline . . . 5
2.3 Producto de matrices y producto de matriz por vector . . . 5
2.4 Tiling y cota inferior de tráfico . . . 5
2.5 Utilización efectiva . . . 6

3 Algoritmos conscientes de la E/S: FlashAttention **7**
3.1 Fusión de kernels . . . 7
3.2 La atención estándar y su tráfico de memoria . . . 7
3.3 La softmax en línea . . . 8
3.4 El algoritmo FlashAttention . . . 8

4 Precisión numérica y precisión mixta **10**
4.1 Formatos de coma flotante . . . 10
4.2 Error de redondeo . . . 11
4.3 La receta de la precisión mixta . . . 11
4.4 Cuantización . . . 12

5 Entrenamiento distribuido **13**
5.1 Operaciones colectivas . . . 13
5.2 La ley de Amdahl . . . 14
5.3 Paralelismo de datos . . . 14
5.4 Paralelismo de tensores . . . 14
5.5 Paralelismo de etapas . . . 15
5.6 ZeRO: partición de los estados del entrenamiento . . . 15
5.7 Recomputación de activaciones . . . 15

6 Coste, energía y límites físicos **17**

6.1 Duración y coste de un entrenamiento 17

6.2 Energía, potencia y refrigeración 17

6.3 Operar frente a mover datos 18

6.4 El límite de Landauer 18

Bibliografía **19**

Tema 1

Cómputo matricial y aceleradores

Casi todo el cómputo de un modelo de lenguaje consiste en productos de matrices densas. Este tema cuenta con precisión esas operaciones, deduce la regla de seis operaciones por parámetro y por token del entrenamiento y describe los dos diseños de hardware que las ejecutan de forma eficiente: la GPU, con sus núcleos tensoriales, y la matriz sistólica de la TPU. Se siguen los capítulos 4 y 7 de Hennessy y Patterson [10], la clase sobre GPU de CMU 15-418/618 [4], la contabilidad de recursos de la tarea 1 de CS336 [5, §3.5], el artículo de Kaplan *et al.* [17, §2.1] y el de Jouppi *et al.* [16].

1.1 Contar operaciones

Definición 1.1 (FLOP). Se llama *operación en coma flotante* (FLOP, del inglés *floating-point operation*) a una suma o a una multiplicación de dos números reales representados en coma flotante. Una multiplicación seguida de una suma acumulada, $c \leftarrow c + ab$ (operación MAC o FMA), cuenta como dos FLOP. El *rendimiento* de un dispositivo se mide en FLOP por segundo (FLOP/s).

Conviene distinguir *FLOP* (cantidad de operaciones) de *FLOP/s* (velocidad). En este módulo, 1 TFLOP/s $= 10^{12}$ FLOP/s.

Proposición 1.2 (Coste del producto de matrices). *Sean $A \in \mathbb{R}^{m\times k}$ y $B \in \mathbb{R}^{k\times n}$. El cálculo directo de $C = AB$ requiere $2mkn$ FLOP.*

Observación 1.3. Estrictamente bastan $k-1$ sumas por elemento, pero el convenio $2mkn$ es el habitual [5, §3.5] y coincide con el número de operaciones MAC multiplicado por dos. Algoritmos como el de Strassen reducen el exponente asintótico, pero no se usan en el entrenamiento de redes porque son menos estables numéricamente y peor adaptados al hardware.

1.2 La regla de seis operaciones por parámetro y token

Proposición 1.5 (Coste de entrenamiento de una capa lineal). *Sea una capa lineal $Y = XW$ con $X \in \mathbb{R}^{T\times a}$ y $W \in \mathbb{R}^{a\times b}$, y sea $G = \partial\mathcal{L}/\partial Y \in \mathbb{R}^{T\times b}$ el gradiente que llega a su salida. La pasada hacia delante cuesta $2Tab$ FLOP y la pasada hacia atrás, que calcula $\partial\mathcal{L}/\partial X$ y $\partial\mathcal{L}/\partial W$, cuesta $4Tab$ FLOP. En total, 6 FLOP por parámetro y por token.*

Corolario 1.6 (Regla $C \approx 6ND$). *Si casi todos los parámetros de un modelo pertenecen a capas lineales y se desprecian las operaciones que no multiplican pesos (normalizaciones, activaciones, producto $QK^\top$ de la atención), entrenar un modelo de N parámetros con D tokens cuesta $C \approx 6ND$ FLOP: $2ND$ en la pasada hacia delante y $4ND$ en la pasada hacia atrás.*

La aproximación es la de Kaplan *et al.* [17, §2.1 y tabla 1], que estiman la pasada hacia delante en $2N + 2n_{\text{capas}}n_{\text{ctx}}d$ FLOP por token; el segundo término, debido a la atención, es pequeño cuando $n_{\text{ctx}} \ll 12d$. Los parámetros de la tabla de *embeddings* de entrada no intervienen en productos de matrices (se leen por índice) y suelen excluirse de N.

1.3 Arquitectura de una GPU

Una CPU dedica la mayor parte de su superficie a hacer rápido un único hilo de ejecución: cachés grandes, predicción de saltos, ejecución fuera de orden. Una GPU invierte el compromiso y dedica la superficie a muchas unidades aritméticas sencillas que ejecutan la misma instrucción sobre datos distintos [10, cap. 4].

Definición 1.7 (Modelo de ejecución SIMT). Una GPU de NVIDIA está formada por *multiprocesadores de streaming* (SM). Un programa para GPU (*kernel*) se lanza como una rejilla de *bloques de hilos*; cada bloque se asigna a un SM y sus hilos pueden cooperar a través de una memoria compartida del SM. El SM ejecuta los hilos en grupos de 32, llamados *warps*, que avanzan con la misma instrucción sobre datos distintos (*single instruction, multiple threads*).

Si los hilos de un warp toman ramas distintas de una condición, el SM ejecuta las dos ramas una tras otra (*divergencia*), y el rendimiento cae. Mientras un warp espera un dato de memoria, el SM ejecuta otro: la GPU oculta la latencia de la memoria con paralelismo, no con cachés [4]. La A100 tiene 108 SM, cada uno con un banco de registros de 256 KB y 192 KB de memoria L1 y compartida [23].

Definición 1.8 (Núcleo tensorial). Un *núcleo tensorial* (*tensor core*) es una unidad que ejecuta en una sola instrucción el producto acumulado $D \leftarrow AB+C$ de pequeñas matrices, con A y B en un formato de baja precisión (fp16, bf16, fp8) y el acumulador en mayor precisión (normalmente fp32).

Los núcleos tensoriales aparecieron en la arquitectura Volta (2017). En la A100, el rendimiento máximo en bf16 con núcleos tensoriales es de 312 TFLOP/s, frente a 19,5 TFLOP/s en fp32 con las unidades ordinarias [23]: un factor $312/19{,}5 = 16$. Un programa que no expresa su cálculo como productos de matrices en baja precisión desaprovecha casi todo el chip.

1.4 Matrices sistólicas

Definición 1.9 (Matriz sistólica). Una *matriz sistólica* de $n \times n$ es una rejilla de celdas MAC conectadas solo con sus vecinas. En cada ciclo de reloj, cada celda recibe un valor por la izquierda y otro por arriba, acumula su producto y los reenvía a la derecha y hacia abajo. En la variante *de salida estacionaria*, la celda (i, j) acumula el elemento C_{ij} del producto.

El nombre alude al latido del corazón: los datos recorren la rejilla a ritmo constante. La ventaja es la reutilización. Cada elemento de A que entra por el borde izquierdo es usado por las n celdas de su fila sin volver a leerse de memoria, y lo mismo cada elemento de B en su columna. Para ello las filas de A y las columnas de B entran escalonadas, como en la figura 1.1; el ejercicio 1.5 demuestra que cada celda recibe los factores correctos en el mismo ciclo.

Observación 1.11. Los núcleos tensoriales de una GPU y la matriz sistólica de una TPU son dos soluciones al mismo problema: aumentar el número de operaciones por cada dato leído. El tema 2 cuantifica esta idea con la intensidad aritmética. El capítulo 7 de Hennessy y Patterson [10] presenta la TPU como ejemplo de *arquitectura de dominio específico*: renuncia a la generalidad a cambio de eficiencia en una sola operación.

Tema 2

Jerarquía de memoria y modelo roofline

Calcular es barato; traer los datos es caro. Este tema describe la jerarquía de memoria de un acelerador, define la intensidad aritmética de una operación y demuestra la cota del modelo *roofline*, que decide si una operación está limitada por el cómputo o por la memoria. Después aplica el modelo al producto de matrices, al producto de matriz por vector y a la generación de texto, y analiza el *tiling* como técnica para aumentar la intensidad. Se siguen el artículo de Williams, Waterman y Patterson [27], el capítulo 4 de Hennessy y Patterson [10], el capítulo «All About Rooflines» de *How to Scale Your Model* [1], la clase 6 de CS336 [9] y el resultado de Hong y Kung [13].

2.1 La jerarquía de memoria

Cada nivel de memoria de un acelerador es más grande y más lento que el anterior. La cuadro 2.1 resume los niveles de la A100; las cifras son aproximadas y proceden de la hoja de especificaciones y de la guía de ajuste de NVIDIA [23].

Nivel	Capacidad	Ancho de banda	Uso
Registros	256 KB por SM	máximo	operandos de cada instrucción
SRAM (L1 y memoria compartida)	192 KB por SM ($\approx$ 20 MB en total)	decenas de TB/s agregados	bloques de datos gestionados por el programa (*tiling*)
Caché L2	40 MB	intermedio	compartida por todos los SM
HBM	80 GB	$\approx$ 2 TB/s	pesos, activaciones y estado del optimizador
NVLink (entre GPU de un nodo)	—	600 GB/s	comunicación del paralelismo
Red entre nodos (InfiniBand)	—	decenas de GB/s por GPU	el enlace más lento

Cuadro 2.1: Jerarquía de memoria y de comunicación de una A100 SXM de 80 GB (valores aproximados).

La HBM (*High Bandwidth Memory*) es una pila de chips de DRAM montada junto al procesador: grande, pero lejana. La SRAM está dentro de cada SM: diminuta, pero muy rápida. Un *kernel* eficiente carga un bloque de datos de la HBM a la SRAM, realiza con él todas las operaciones posibles y solo entonces escribe el resultado.

2.2 Intensidad aritmética y modelo roofline

Definición 2.1 (Intensidad aritmética). Sea un *kernel* que realiza F FLOP y transfiere Q bytes entre la memoria principal (HBM) y el procesador. Su *intensidad aritmética* es $I = F/Q$, en FLOP por byte.

Definición 2.2 (Modelo roofline). Sea un dispositivo con rendimiento máximo P_{pico} (FLOP/s) y ancho de banda de memoria B_{mem} (bytes/s). Su *intensidad crítica* es $I^* = P_{\text{pico}}/B_{\text{mem}}$. El *modelo roofline* [27] asigna a un *kernel* de intensidad I el rendimiento alcanzable

$$P(I) = \min\bigl(P_{\text{pico}},\ I\,B_{\text{mem}}\bigr).$$

Se dice que el *kernel* está *limitado por la memoria* si $I < I^*$ y *limitado por el cómputo* si $I > I^*$.

Teorema 2.3 (Cota del roofline). *Sea un* kernel *que realiza F FLOP y transfiere Q bytes en un dispositivo con parámetros P_{pico} y B_{mem}. Su tiempo de ejecución cumple*

$$t \;\geq\; \max\Bigl(\frac{F}{P_{\text{pico}}},\ \frac{Q}{B_{\text{mem}}}\Bigr), \qquad \text{y por tanto} \qquad \frac{F}{t} \;\leq\; \min\bigl(P_{\text{pico}},\ I\,B_{\text{mem}}\bigr), \quad I = \frac{F}{Q}.$$

La cota se alcanza solo si el cálculo y la transferencia se solapan por completo; si se ejecutasen en serie, el tiempo sería la suma $F/P_{\text{pico}} + Q/B_{\text{mem}}$, que es como mucho el doble del máximo. En ejes logarítmicos, $P(I)$ es una rampa de pendiente 1 seguida de un techo horizontal: de ahí el nombre (figura 2.1).

2.3 Producto de matrices y producto de matriz por vector

Proposición 2.5 (Intensidad del producto de matrices). *Sea $C = AB$ con $A \in \mathbb{R}^{m\times k}$, $B \in \mathbb{R}^{k\times n}$, y sean s los bytes por número. Si cada matriz se lee o escribe una sola vez, la intensidad es*

$$I = \frac{2mkn}{s\,(mk + kn + mn)}.$$

En particular, para matrices cuadradas ($m = k = n$), $I = 2n/(3s)$, que en bf16 ($s = 2$) vale $n/3$.

Corolario 2.6 (Producto de matriz por vector). *Si $n = 1$ (un solo vector) y $m, k \gg 1$, la intensidad es $I \approx 2/s$, que en bf16 vale 1 FLOP/byte. Con b vectores ($n = b \ll m, k$), $I \approx 2b/s$, que en bf16 vale b.*

En el producto de matriz por vector cada peso leído se usa en una sola multiplicación. En la A100, $P \leq 1 \cdot 2 = 2$ TFLOP/s, un 0,6 % del máximo. Por eso la generación de texto token a token está limitada por la memoria (módulo 19): con lote 1, cada token exige un producto de matriz por vector con cada matriz de pesos.

2.4 Tiling y cota inferior de tráfico

Un producto de matrices grande no cabe en la SRAM. El tráfico efectivo con la HBM depende de cómo se recorra.

Proposición 2.8 (Tráfico del producto por bloques). *Sean $A, B \in \mathbb{R}^{n\times n}$ y sea b un divisor de n. El algoritmo que, para cada bloque $b \times b$ de C, recorre los n/b pares de bloques $b\times b$ de A y de B que contribuyen a él, los carga en la SRAM y acumula su producto, transfiere*

$$Q(b) = s\Big(\frac{2n^3}{b} + n^2\Big) \text{ bytes},$$

necesita una SRAM con capacidad para $3b^2$ números y tiene intensidad $I(b) = 2n^3/Q(b) \approx b/s$ si $n \gg b$.

La intensidad crece linealmente con el tamaño del bloque. Si la SRAM admite M números, la condición $3b^2 \le M$ da $b \le \sqrt{M/3}$ y un tráfico del orden de $n^3/\sqrt{M}$. Hong y Kung demostraron que este orden es óptimo:

Teorema 2.9 (Hong y Kung, 1981). *Todo algoritmo que calcula el producto de dos matrices $n\times n$ por el método clásico de n^3 productos con una memoria rápida de M números transfiere $\Omega\big(n^3/\sqrt{M}\big)$ números entre la memoria rápida y la lenta.*

La demostración, mediante el juego de los guijarros rojos y azules, puede consultarse en [13]. El ejercicio 2.6 pide aplicar la proposición 2.8 a la SRAM de un SM.

Observación 2.10. La proposición 2.8 describe un solo nivel de la jerarquía. En una GPU hay varios: registros, SRAM y caché L2. La caché L2 (40 MB en la A100) permite que bloques de hilos vecinos reutilicen los mismos bloques de A y B, de modo que el tráfico real con la HBM es menor que el que da la proposición con el b de la SRAM. El principio es el mismo en cada nivel: cuanto mayor es el bloque que cabe, mayor es la reutilización.

2.5 Utilización efectiva

Definición 2.11 (Utilización de FLOP del modelo). Sea un entrenamiento que procesa τ tokens por segundo con un modelo de N parámetros en n_{d} dispositivos de rendimiento máximo P_{pico}. Su *utilización de FLOP del modelo* es [3, §4.1 y apéndice B]

$$\mathrm{MFU} = \frac{6N\tau}{n_{\mathrm{d}}\,P_{\mathrm{pico}}}.$$

El numerador cuenta solo los FLOP «útiles» del corolario 1.6; la recomputación de activaciones (tema 5) no suma. Los entrenamientos grandes bien optimizados se sitúan típicamente entre el 30 % y el 60 % de MFU (PaLM 540B declaró un 46,2 % [3, §4.1]); el resto se pierde en comunicación, en operaciones limitadas por la memoria y en esperas.

Tema 3

Algoritmos conscientes de la E/S: FlashAttention

El tema 2 mostró que una operación de baja intensidad aritmética está limitada por la memoria, por rápido que sea el cómputo. Este tema aplica esa idea a la atención, cuya implementación directa escribe y lee de la HBM una matriz de tamaño cuadrático en la longitud de la secuencia. Se presentan la fusión de *kernels*, el análisis de entrada y salida (E/S) de la atención estándar, la softmax en línea y el algoritmo FlashAttention, que calcula la atención exacta sin materializar esa matriz. Se siguen el artículo de Dao *et al.* [7, §2–3], el de Milakov y Gimelshein [21], la clase 6 de CS336 [9] y el §4 de la tarea 2 de ese curso [6]. En este tema, n es la longitud de la secuencia, d la dimensión de cada cabeza y M la capacidad de la SRAM, medida en elementos.

3.1 Fusión de kernels

Definición 3.1 (Operación elemento a elemento). Una operación $y = g(x)$ sobre un tensor de L elementos es *elemento a elemento* si cada y_i depende solo de x_i. Ejemplos: las activaciones GELU y SiLU, la suma de un sesgo, la multiplicación por un escalar.

Proposición 3.2 (Efecto de la fusión). *Sea una cadena de r operaciones elemento a elemento $g_r \circ \cdots \circ g_1$ sobre un tensor de L elementos de s bytes, cada una con f FLOP por elemento. Ejecutada como r* kernels *separados transfiere $2rLs$ bytes; fusionada en un solo* kernel *transfiere $2Ls$ bytes. La intensidad pasa de $f/(2s)$ a $rf/(2s)$.*

Los compiladores como `torch.compile` y los lenguajes como Triton automatizan esta fusión [9]. La atención no es una cadena elemento a elemento, porque la softmax normaliza por filas, pero la idea es la misma: no escribir resultados intermedios en la HBM.

3.2 La atención estándar y su tráfico de memoria

Sean $Q, K, V \in \mathbb{R}^{n \times d}$. La atención de una cabeza (módulo 13) calcula

$$S = QK^\top \in \mathbb{R}^{n \times n}, \qquad P = \mathrm{softmax}(S) \text{ (por filas)}, \qquad O = PV \in \mathbb{R}^{n \times d},$$

donde el factor $1/\sqrt{d}$ se supone incorporado en Q.

Proposición 3.3 (Tráfico de la atención estándar). *La implementación en tres* kernels *(calcular y escribir S; leer S y escribir P; leer P y V y escribir O) transfiere $4n^2 + 4nd$ elementos entre la HBM y el procesador, y necesita n^2 elementos de memoria adicional para S o P.*

Cuando $n \gg d$, el término $4n^2$ domina. La softmax tiene intensidad baja (unos pocos FLOP por elemento), de modo que el segundo *kernel* está limitado por la memoria. Con $n = 4096$ en bf16, S ocupa $4096^2 \cdot 2$ bytes $= 32$ MiB por cabeza; con 32 cabezas, 1 GiB por capa y por secuencia. Además, la retropropagación necesita P, que debe guardarse (ejercicio 3.5).

3.3 La softmax en línea

La softmax de una fila $x \in \mathbb{R}^n$ se calcula de forma estable restando el máximo (módulo 1): $\mathrm{softmax}(x)_i = e^{x_i - m}/\ell$, con $m = \max_j x_j$ y $\ell = \sum_j e^{x_j - m}$. Las dos cantidades m y ℓ pueden obtenerse por bloques [21].

Definición 3.4 (Estadísticos de un bloque). Sea $\mathcal{B} \subseteq \{1, \dots, n\}$ no vacío. Sus *estadísticos* son

$$m_{\mathcal{B}} = \max_{j \in \mathcal{B}} x_j, \qquad \ell_{\mathcal{B}} = \sum_{j \in \mathcal{B}} e^{x_j - m_{\mathcal{B}}}.$$

Lema 3.5 (Combinación de estadísticos). *Sean $\mathcal{B}_1$ y $\mathcal{B}_2$ bloques disjuntos con estadísticos (m_1, ℓ_1) y (m_2, ℓ_2), y sea $m = \max(m_1, m_2)$. Entonces los estadísticos de $\mathcal{B}_1 \cup \mathcal{B}_2$ son*

$$m_{\mathcal{B}_1 \cup \mathcal{B}_2} = m, \qquad \ell_{\mathcal{B}_1 \cup \mathcal{B}_2} = \ell_1 e^{m_1 - m} + \ell_2 e^{m_2 - m}.$$

Además, ningún factor $e^{m_i - m}$ es mayor que 1.

Proposición 3.6 (Combinación de salidas). *Sean $v_1, \dots, v_n \in \mathbb{R}^d$ y, para cada bloque, $o_{\mathcal{B}} = \ell_{\mathcal{B}}^{-1} \sum_{j \in \mathcal{B}} e^{x_j - m_{\mathcal{B}}} v_j$ la salida de la atención restringida al bloque. Con la notación del lema 3.5 y $\ell = \ell_1 e^{m_1 - m} + \ell_2 e^{m_2 - m}$,*

$$o_{\mathcal{B}_1 \cup \mathcal{B}_2} = \frac{\ell_1 e^{m_1 - m}\, o_{\mathcal{B}_1} + \ell_2 e^{m_2 - m}\, o_{\mathcal{B}_2}}{\ell}.$$

3.4 El algoritmo FlashAttention

FlashAttention [7, algoritmo 1] divide Q en T_r bloques de B_r filas y K, V en T_c bloques de B_c filas, con

$$B_c = \left\lceil \frac{M}{4d} \right\rceil, \qquad B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil,\ d \right), \qquad T_r = \frac{n}{B_r}, \qquad T_c = \frac{n}{B_c},$$

de forma que los bloques en uso caben en una SRAM de tamaño $O(M)$. Para cada fila de Q mantiene en la HBM la salida parcial normalizada y los estadísticos (m, ℓ), y los actualiza con el lema 3.5 y la proposición 3.6 cada vez que procesa un nuevo bloque de claves (figuras 3.1 y 3.2).

Teorema 3.8 (Exactitud y memoria; Dao *et al.*, teorema 1). *El algoritmo de la figura 3.1 devuelve exactamente $O = \mathrm{softmax}(QK^\top)V$ (salvo redondeo), con $O(n^2 d)$ FLOP y $O(n)$ memoria adicional en HBM aparte de la entrada y la salida.*

Proposición 3.9 (Tráfico de FlashAttention). *El algoritmo de la figura 3.1 transfiere*

$$Q_{\mathrm{FA}} = (nd + 2n) + 2nd + T_c\,(3nd + 4n)$$

elementos entre la HBM y el procesador. Como $T_c \approx 4nd/M$, el término dominante es $12\, n^2 d^2 / M$.

Dao *et al.* [7, teorema 2] enuncian el resultado asintótico: la atención estándar requiere $\Theta(nd + n^2)$ accesos a la HBM y FlashAttention $\Theta(n^2 d^2 M^{-1})$. Con valores típicos (d entre 64 y 128, M de unos 10^5 elementos), d^2/M es mucho menor que 1 y el ahorro es grande. También demuestran que ningún algoritmo exacto puede hacer $o(n^2 d^2 M^{-1})$ accesos para todo $M \in [d, nd]$ [7, proposición 3].

Observación 3.10 (Pasada hacia atrás y recomputación). La retropropagación de la atención necesita P. FlashAttention no la guarda: almacena solo (m, ℓ) y, en la pasada hacia atrás, recalcula cada bloque S_{ij} en la SRAM a partir de Q_i y K_j. Realiza más FLOP que la implementación estándar, pero muchos menos accesos a la HBM, y resulta más rápida [7, §3.1]. Es el principio del roofline: en una operación limitada por la memoria, recalcular es más barato que leer.

Tema 4

Precisión numérica y precisión mixta

Los núcleos tensoriales son mucho más rápidos en 16 u 8 bits que en 32, y cada bit que se ahorra reduce también la memoria y el tráfico. El precio es un mayor error de redondeo y un rango más estrecho. Este tema formaliza los formatos de coma flotante, acota el error de redondeo y de la suma acumulada, y justifica con esas cotas cada elemento de la receta de entrenamiento en precisión mixta. Se siguen los capítulos 2 a 4 de Higham [11], el artículo de Micikevicius *et al.* [19, §3], la propuesta de formatos fp8 [20], las clases de cuantización de MIT 6.5940 [8] y el §2.1 de la tarea 2 de CS336 [6]. El módulo 11 introdujo estos formatos de forma descriptiva; aquí se demuestran sus propiedades.

4.1 Formatos de coma flotante

Definición 4.1 (Formato binario de coma flotante). Un formato binario con e bits de exponente y p bits de mantisa representa los números *normales*

$$x = (-1)^{\sigma}\,(1, f_1 f_2 \ldots f_p)_2 \cdot 2^{E}, \qquad E_{\min} \le E \le E_{\max},$$

con $\sigma \in \{0,1\}$ y $f_i \in \{0,1\}$. En los formatos IEEE 754 el exponente se guarda con un sesgo $b = 2^{e-1} - 1$, y $E_{\min} = 1 - b$, $E_{\max} = b$; los códigos de exponente extremos se reservan para el cero, los números subnormales, los infinitos y los NaN.

Proposición 4.2 (Rango y separación). *En el formato de la definición 4.1:*

(i) el mayor número representable es $(2 - 2^{-p})\,2^{E_{\max}}$ *y el menor normal positivo,* $2^{E_{\min}}$*;*

(ii) en cada intervalo $[2^{E}, 2^{E+1})$ *con* $E_{\min} \le E \le E_{\max}$ *hay* 2^{p} *números representables equiespaciados, separados por* $\mathrm{ulp} = 2^{E-p}$*;*

(iii) la separación entre 1 *y el siguiente representable es el* épsilon de la máquina, $\varepsilon = 2^{-p}$.

Formato	bits (s-e-m)	máximo	menor normal	$\varepsilon = 2^{-p}$	uso típico
fp32	1-8-23	$3{,}40 \cdot 10^{38}$	$1{,}18 \cdot 10^{-38}$	$1{,}19 \cdot 10^{-7}$	copia maestra, acumulación
fp16	1-5-10	65 504	$6{,}10 \cdot 10^{-5}$	$9{,}77 \cdot 10^{-4}$	productos
bf16	1-8-7	$3{,}39 \cdot 10^{38}$	$1{,}18 \cdot 10^{-38}$	$7{,}81 \cdot 10^{-3}$	productos, activaciones
fp8 E5M2	1-5-2	57 344	$6{,}10 \cdot 10^{-5}$	0,25	gradientes
fp8 E4M3	1-4-3	448	$1{,}56 \cdot 10^{-2}$	0,125	pesos, activaciones

Cuadro 4.1: Formatos de coma flotante usados en el entrenamiento. El formato E4M3 no sigue del todo IEEE 754: no tiene infinitos y reutiliza el código de exponente máximo, lo que eleva su máximo de 240 a 448 [20].

El rango depende de los bits de exponente y la precisión, de los de mantisa. bf16 conserva los 8 bits de exponente de fp32 y, por tanto, su rango; fp16 tiene 3 bits más de mantisa, pero su rango es mucho menor. Por debajo del menor normal, los números subnormales extienden el rango con pérdida de precisión: en fp16 el menor positivo es $2^{-24} \approx 5{,}96 \cdot 10^{-8}$.

4.2 Error de redondeo

Definición 4.3 (Redondeo y unidad de redondeo). Sea $\mathrm{fl}(x)$ el número representable más cercano a $x \in \mathbb{R}$ (redondeo al más cercano, con empates al par). La *unidad de redondeo* del formato es $u = \varepsilon/2 = 2^{-p-1}$.

Teorema 4.4 (Error relativo de redondeo). *Si x está en el rango normal del formato, $\mathrm{fl}(x) = x(1+\delta)$ con $|\delta| \le u$.*

La norma IEEE 754 exige que las operaciones básicas se redondeen correctamente: $\mathrm{fl}(a \circ b) = (a \circ b)(1+\delta)$, $|\delta| \le u$, para $\circ \in \{+, -, \times, /\}$ [11, cap. 2]. Es el *modelo estándar* de la aritmética en coma flotante.

Lema 4.5 (Estancamiento). *Sea $s > 0$ representable con $2^E \le s < 2^{E+1}$ y sea $0 \le a < 2^{E-p-1} = \mathrm{ulp}(s)/2$. Entonces $\mathrm{fl}(s + a) = s$.*

Corolario 4.6 (Actualizaciones perdidas). *En bf16, si un peso vale $w = 1$, toda actualización $0 \le \Delta w < 2^{-8} \approx 0{,}0039$ se pierde: $\mathrm{fl}(1 + \Delta w) = 1$.*

Lema 4.7 (Producto de factores de redondeo). *Sean $\delta_1, \dots, \delta_k$ con $|\delta_i| \le u$ y $ku < 1$. Entonces*

$$\left| \prod_{i=1}^{k} (1+\delta_i) - 1 \right| \le \gamma_k, \qquad \gamma_k = \frac{ku}{1-ku}.$$

Teorema 4.8 (Error de la suma recursiva). *Sea $\hat{s}_n$ el resultado de sumar $x_1, \dots, x_n$ en coma flotante en el orden $\hat{s}_1 = x_1$, $\hat{s}_k = \mathrm{fl}(\hat{s}_{k-1} + x_k)$. Si $(n-1)u < 1$ y no hay desbordamiento,*

$$|\hat{s}_n - s_n| \le \gamma_{n-1} \sum_{i=1}^{n} |x_i|, \qquad s_n = \sum_{i=1}^{n} x_i.$$

La cota crece con n. En fp16, $u = 2^{-11} \approx 4{,}9 \cdot 10^{-4}$ y con $n = 1000$ sumandos $\gamma_{999} \approx 0{,}95$: la cota no garantiza ni una cifra. En fp32, $u = 2^{-24} \approx 6 \cdot 10^{-8}$ y $\gamma_{999} \approx 6 \cdot 10^{-5}$. Esta es la razón por la que los núcleos tensoriales acumulan los productos escalares en fp32 aunque los factores estén en 16 bits (ejercicio 4.4).

4.3 La receta de la precisión mixta

Micikevicius *et al.* [19, §3] proponen tres técnicas, que siguen siendo la base de la práctica actual.

(1) *Copia maestra de los pesos en fp32* (§3.1). Los productos usan una copia en 16 bits, pero la actualización $w \leftarrow w - \eta g$ se aplica a una copia en fp32. Por el corolario 4.6, una actualización pequeña frente al peso se perdería en 16 bits; los autores observan que esto ocurre cuando el peso es unas 2^{11} veces mayor que la actualización.

(2) *Escalado de la pérdida* (§3.2). En fp16, los valores menores que 2^{-24} se anulan, y parte de los gradientes de activación caen por debajo. Se multiplica la pérdida por una constante c antes de la retropropagación y se dividen los gradientes por c antes de actualizar.

(3) *Acumulación en fp32* (§3.3). Los productos escalares acumulan en fp32, y las reducciones grandes (softmax, normalizaciones, pérdida) se calculan en fp32.

Proposición 4.9 (El escalado por potencias de 2 es exacto). *Sea $c = 2^k$ con $k \in \mathbb{Z}$, y sea x tal que x y cx están en el rango normal. Entonces* $\mathrm{fl}(cx) = c\,\mathrm{fl}(x)$. *En particular, multiplicar y después dividir por c un número representable no introduce error.*

Como la retropropagación es lineal en la semilla $\partial\mathcal{L}/\partial\mathcal{L} = 1$, multiplicar la pérdida por c multiplica por c todos los gradientes, y por la proposición 4.9 la división final por $c = 2^k$ los restituye sin error adicional, salvo en los valores que se hayan desbordado o anulado. En la práctica se usa un escalado *dinámico*: se duplica c periódicamente y se reduce a la mitad, descartando el paso, cuando aparece un infinito. Con bf16, cuyo rango es el de fp32, el escalado no es necesario, y esa es la razón principal de que bf16 sea hoy el formato habitual de entrenamiento.

Observación 4.10 (Memoria de la receta). Con Adam y precisión mixta, cada parámetro ocupa 2 bytes (pesos en 16 bits) + 2 (gradiente en 16 bits) + 12 (copia maestra, primer y segundo momento en fp32) = 16 bytes [25, §3.1]. El tema 5 estudia cómo repartir esos 16 bytes entre dispositivos.

4.4 Cuantización

El entrenamiento en fp8 y la inferencia en int8 o int4 llevan la idea al extremo. Con tan pocos bits de mantisa, cada tensor necesita su propio factor de escala.

Definición 4.11 (Cuantización lineal simétrica). Sea $x \in \mathbb{R}^L$ y $b \geq 2$ un número de bits. La *cuantización lineal simétrica* de x con b bits usa el paso $\Delta = \max_i |x_i|/(2^{b-1} - 1)$ y los enteros $q_i = \mathrm{round}(x_i/\Delta) \in \{-(2^{b-1} - 1), \ldots, 2^{b-1} - 1\}$. La *descuantización* es $\hat{x}_i = \Delta\, q_i$.

Proposición 4.12. *Con la definición 4.11, $|\hat{x}_i - x_i| \leq \Delta/2$ para todo i.*

El error depende del valor máximo del tensor: un solo valor atípico grande aumenta Δ y degrada la representación de todos los demás. Por eso se cuantiza por canales o por bloques, y se tratan aparte los valores atípicos [8]. El módulo 19 retoma la cuantización para la inferencia.

Tema 5

Entrenamiento distribuido

Un modelo grande no cabe en la memoria de un acelerador, y su entrenamiento tardaría siglos en uno solo. Este tema estudia cómo se reparte el trabajo entre muchos dispositivos. Empieza por las operaciones colectivas de comunicación y su coste, enuncia la ley de Amdahl y analiza las tres formas básicas de paralelismo (de datos, de tensores y de etapas), la partición de estados de ZeRO y la recomputación de activaciones. Se siguen la clase 7 de CS336 [9] y el §8 de su tarea 2 [6], el capítulo 1 de Hennessy y Patterson [10] para la ley de Amdahl, la clase «Interconnection networks» de CMU 15-418/618 [4], los artículos de Patarasuk y Yuan [24], Shoeybi *et al.* [26, §3], Huang *et al.* [15, §2.3], Narayanan *et al.* [22, §2–3], Rajbhandari *et al.* [25, §3–7] y Chen *et al.* [2].

5.1 Operaciones colectivas

Sean n_{d} dispositivos (llamados *rangos*) numerados $0, \dots, n_{\mathrm{d}} - 1$, y sea $x^{(r)}$ un vector de S bytes en el rango r.

Definición 5.1 (Operaciones colectivas). (i) *All-reduce*: al terminar, todos los rangos tienen $\sum_r x^{(r)}$.

(ii) *Reduce-scatter*: se divide cada vector en n_{d} trozos $x_0^{(r)}, \dots, x_{n_{\mathrm{d}}-1}^{(r)}$; al terminar, el rango c tiene solo el trozo c de la suma, $\sum_r x_c^{(r)}$.

(iii) *All-gather*: el rango c tiene un trozo y_c; al terminar, todos los rangos tienen la concatenación $(y_0, \dots, y_{n_{\mathrm{d}}-1})$.

Un all-reduce equivale a un reduce-scatter seguido de un all-gather de los trozos ya sumados [9].

Proposición 5.2 (All-reduce en anillo). *Sean n_{d} rangos conectados en anillo, cada uno capaz de enviar a su vecino a W bytes/s mientras recibe del otro. El algoritmo en anillo realiza un all-reduce de S bytes enviando cada rango $2(n_{\mathrm{d}} - 1)S/n_{\mathrm{d}}$ bytes, en un tiempo (sin contar latencias)*

$$t_{\mathrm{AR}} = \frac{2(n_{\mathrm{d}} - 1)}{n_{\mathrm{d}}} \cdot \frac{S}{W}.$$

Observación 5.3. El volumen $2(n_{\mathrm{d}} - 1)S/n_{\mathrm{d}}$ es prácticamente independiente de n_{d} y tiende a $2S$. Patarasuk y Yuan [24] demostraron que, sin compresión de los datos, en todo algoritmo de all-reduce algún rango debe comunicar al menos ese volumen: el anillo es óptimo en ancho de banda. El algoritmo tiene, en cambio, $2(n_{\mathrm{d}} - 1)$ pasos, cada uno con su latencia; para tensores pequeños o muchos rangos se prefieren algoritmos en árbol.

5.2 La ley de Amdahl

Teorema 5.4 (Ley de Amdahl). *Sea un programa cuyo tiempo en un dispositivo es t_1, del que una fracción $f \in [0,1]$ no puede paralelizarse y el resto se reparte perfectamente entre n_{d} dispositivos. Su aceleración es*

$$A(n_{\mathrm{d}}) = \frac{t_1}{t_{n_{\mathrm{d}}}} = \frac{1}{f + (1-f)/n_{\mathrm{d}}} \;\leq\; \frac{1}{f}.$$

En el entrenamiento distribuido, el papel de la fracción serie lo desempeñan la comunicación que no se solapa con el cálculo y las esperas. Una fracción del 5 % limita la aceleración a 20, sea cual sea el número de GPU (ejercicio 5.4).

5.3 Paralelismo de datos

Proposición 5.5 (Equivalencia del paralelismo de datos). *Sea un lote de B ejemplos con pérdida media $\mathcal{L}(\theta) = B^{-1}\sum_{i=1}^{B} \ell_i(\theta)$, repartido en n_{d} trozos de B/n_{d} ejemplos, y sea g_r el gradiente de la pérdida media del trozo r. Entonces*

$$\nabla \mathcal{L}(\theta) = \frac{1}{n_{\mathrm{d}}} \sum_{r=0}^{n_{\mathrm{d}}-1} g_r.$$

En el *paralelismo de datos*, cada rango guarda una copia completa del modelo, procesa su trozo del lote y, antes de actualizar, se hace un all-reduce de los gradientes. Por la proposición 5.5, el resultado es idéntico al de un solo dispositivo con el lote completo. Con 10^9 parámetros en bf16 ($S = 2 \cdot 10^9$ bytes), $n_{\mathrm{d}} = 8$ y $W = 100$ GB/s, la proposición 5.2 da $t_{\mathrm{AR}} = 2 \cdot (7/8) \cdot 2 \cdot 10^9 / 10^{11} = 35$ ms por paso. Las implementaciones reales solapan esa comunicación con la pasada hacia atrás, enviando los gradientes de las últimas capas mientras se calculan los de las primeras.

5.4 Paralelismo de tensores

Proposición 5.6 (Partición del MLP; Shoeybi *et al.*). *Sea el MLP $Z = \mathrm{GELU}(XA)\,B$ con $X \in \mathbb{R}^{T\times d}$, $A \in \mathbb{R}^{d\times h}$, $B \in \mathbb{R}^{h\times d}$. Si se parte A por columnas, $A = [A_1 \; A_2]$, y B por filas, $B = \left[\begin{smallmatrix} B_1 \\ B_2 \end{smallmatrix}\right]$, con bloques compatibles, entonces*

$$Z = \mathrm{GELU}(XA_1)\,B_1 + \mathrm{GELU}(XA_2)\,B_2.$$

Por tanto, dos rangos que guardan (A_1, B_1) y (A_2, B_2) calculan cada uno un sumando sin comunicarse, y basta un all-reduce de Z al final.

La elección de los cortes no es arbitraria. Si se partiera A por filas (y X por columnas), $XA = X_1A_1 + X_2A_2$ y haría falta sumar antes de aplicar GELU, porque en general $\mathrm{GELU}(u+v) \neq \mathrm{GELU}(u) + \mathrm{GELU}(v)$: un punto de sincronización más [26, §3]. En la atención, las proyecciones Q, K y V se parten por columnas, es decir, por cabezas, y la proyección de salida por filas. Cada bloque Transformer requiere así dos all-reduce en la pasada hacia delante y dos en la pasada hacia atrás, sobre tensores de tamaño proporcional a $b \cdot T \cdot d$ (microlote por longitud por dimensión) [22, §3.2]. Como esta comunicación ocurre en cada capa, el paralelismo de tensores se usa dentro de un nodo, sobre NVLink.

5.5 Paralelismo de etapas

Definición 5.7 (Pipeline de GPipe). En el *paralelismo de etapas* (*pipeline*) se reparten las capas en p_e etapas consecutivas, una por rango, y el lote en m microlotes. En el esquema de GPipe [15], la etapa s procesa el microlote j hacia delante en cuanto ha terminado el microlote $j-1$ y la etapa $s-1$ le ha entregado las activaciones del microlote j; la pasada hacia atrás se hace después, en orden inverso.

Proposición 5.8 (Burbuja del pipeline). *Si cada microlote tarda τ en cada etapa (solo pasada hacia delante), el pipeline de GPipe con p_e etapas y m microlotes tarda $(m+p_e-1)\tau$, y la fracción del tiempo en que cada rango está ocioso es*

$$\beta_{\text{tot}} = \frac{p_e - 1}{m + p_e - 1}.$$

La demostración se pide en el ejercicio 5.7. Narayanan *et al.* [22, §2.2.1] miden la burbuja respecto al tiempo ideal $m\tau$ y no respecto al total: $\beta = (p_e-1)/m$. Ambas medidas se relacionan por $\beta_{\text{tot}} = \beta/(1+\beta)$. Huang *et al.* [15, §2.3] observan que la burbuja es despreciable con $m \geq 4p_e$. El esquema 1F1B, que alterna una pasada hacia delante y una hacia atrás, tiene la misma burbuja, pero limita a p_e el número de microlotes cuyas activaciones deben guardarse a la vez, frente a m en GPipe; la variante intercalada, con v trozos de capas por rango, reduce la burbuja a $(p_e-1)/(vm)$ a cambio de multiplicar por v la comunicación [22, §2.2.2].

5.6 ZeRO: partición de los estados del entrenamiento

En el paralelismo de datos, cada rango guarda una copia completa de los 16 bytes por parámetro de la precisión mixta con Adam (tema 4): 2 de pesos, 2 de gradientes y $K = 12$ de estado del optimizador. ZeRO [25] elimina esa redundancia.

Proposición 5.9 (Memoria de ZeRO). *Sea un modelo de Ψ parámetros entrenado con paralelismo de datos en n_d rangos, con 2 bytes de pesos, 2 de gradientes y $K = 12$ de estado del optimizador por parámetro. La memoria de estados por rango es:*

Variante	*Qué se reparte*	*Memoria por rango*
Sin ZeRO	*nada*	16Ψ
ZeRO-1	*estado del optimizador*	$4\Psi + 12\Psi/n_d$
ZeRO-2	*además, gradientes*	$2\Psi + 14\Psi/n_d$
ZeRO-3	*además, pesos*	$16\Psi/n_d$

En ZeRO-3, cada rango reúne con un all-gather los pesos de una capa justo antes de usarla y los descarta después; los gradientes se reducen con un reduce-scatter, de modo que cada rango recibe solo su trozo. El volumen de comunicación por paso es 2Ψ en ZeRO-1 y ZeRO-2, igual que en el paralelismo de datos, y 3Ψ en ZeRO-3, un 50 % más [25, §7]. FSDP (*Fully Sharded Data Parallel*) es la implementación de ZeRO-3 en PyTorch.

5.7 Recomputación de activaciones

La tabla de la proposición 5.9 no incluye las activaciones que la retropropagación necesita (módulo 10), que crecen con el lote y la longitud de la secuencia.

Proposición 5.11 (Recomputación por segmentos). *Sea una red de L capas en cadena cuyas activaciones ocupan una unidad de memoria cada una. Si se divide en k segmentos de L/k capas, se guardan solo las activaciones en las fronteras de los segmentos y, durante la pasada hacia atrás, se recalcula cada segmento a partir de su frontera, la memoria máxima de activaciones es $k + L/k$, a cambio de una pasada hacia delante adicional. Con $k = \sqrt{L}$ la memoria es $2\sqrt{L}$, mínima entre todas las elecciones de k.*

El resultado es el de Chen *et al.* [2]. Con $L = 64$ capas, la memoria de activaciones baja de 64 a 16 unidades a cambio de un tercio más de cómputo (una pasada hacia delante adicional sobre las tres equivalentes del corolario 1.6). La recomputación no cuenta en el MFU (definición 2.11).

Observación 5.12 (Paralelismo 3D). Los entrenamientos grandes combinan las tres formas: $n_{\mathrm{d}} = d_{\mathrm{dp}} \cdot t \cdot p_{\mathrm{e}}$, con t rangos de paralelismo de tensores dentro de cada nodo (comunicación por capa, sobre NVLink), p_{e} etapas de pipeline entre nodos (comunicación punto a punto de activaciones) y d_{dp} réplicas de paralelismo de datos, a menudo con ZeRO, en el nivel exterior (comunicación de gradientes una vez por paso). Narayanan *et al.* [22, §3] analizan esta composición y recomiendan limitar t al tamaño del nodo.

Tema 6

Coste, energía y límites físicos

Los temas anteriores han tratado el rendimiento de un dispositivo y de un conjunto de ellos. Este tema convierte esas magnitudes en las que deciden un proyecto real: duración, coste económico, energía y refrigeración. Después compara la energía de las operaciones con la del movimiento de datos y con el límite físico de Landauer para el borrado de información. Se siguen el §2.1 de Kaplan *et al.* [17], el §4.1 y el apéndice B de Chowdhery *et al.* [3], la tarea 1 de CS336 [5, §4.3], el trabajo de Horowitz [14], el capítulo 6 de Hennessy y Patterson [10] sobre centros de datos y el artículo de Landauer [18].

6.1 Duración y coste de un entrenamiento

Proposición 6.1 (Duración de un entrenamiento). *Sea un entrenamiento de C FLOP en n_{d} dispositivos de rendimiento máximo P_{pico} con utilización* MFU*. Su duración es*

$$t = \frac{C}{n_{\mathrm{d}}\, P_{\mathrm{pico}}\, \mathrm{MFU}},$$

y consume $n_{\mathrm{d}} t$ dispositivos-segundo, que no dependen de n_{d} si el MFU se mantiene.

El coste económico es, en primera aproximación, el número de GPU-hora por el precio de la GPU-hora. Duplicar el número de dispositivos reduce la duración a la mitad sin cambiar el coste, mientras el MFU no caiga; por la ley de Amdahl (teorema 5.4) y el coste creciente de la comunicación, en la práctica cae.

6.2 Energía, potencia y refrigeración

Definición 6.3 (PUE). La *eficiencia en el uso de la energía* (PUE, *Power Usage Effectiveness*) de un centro de datos es el cociente entre la energía total que consume y la que consumen los equipos de cómputo. Siempre es PUE ≥ 1; el exceso corresponde a refrigeración, conversión eléctrica y otros servicios [10, cap. 6].

La energía de un entrenamiento es, aproximadamente, $E \approx (\text{GPU-hora}) \cdot P_{\mathrm{GPU}} \cdot \mathrm{PUE}$, donde P_{GPU} es la potencia media por GPU (la máxima de la A100 SXM es de 400 W [23]). Esta estimación ignora las CPU, la red y el almacenamiento.

Casi toda la energía eléctrica se disipa como calor, que debe evacuarse. Si un fluido de calor específico c entra a una temperatura y sale ΔT más caliente, el calor que extrae por unidad de tiempo es

$$\dot{Q} = \dot{m}\, c\, \Delta T,$$

con $\dot{m}$ el caudal másico. Para el agua, $c \approx 4186$ J/(kg·K); para el aire, $c \approx 1005$ J/(kg·K), con una densidad unas 800 veces menor. Evacuar los $1024 \cdot 400 = 409{,}6$ kW de las GPU del

ejemplo 6.2 con agua que se calienta 10 K exige $\dot{m} = 409\,600/(4186 \cdot 10) \approx 9{,}8$ kg/s, casi diez litros por segundo. Por eso los sistemas actuales de alta densidad pasan de la refrigeración por aire a la refrigeración líquida directa sobre el chip (ejercicio 6.4).

6.3 Operar frente a mover datos

Horowitz [14] recopiló la energía de operaciones elementales en una tecnología de 45 nm. Las cifras, de orden de magnitud, muestran que una suma en fp32 cuesta unos 0,9 pJ y una multiplicación en fp32, unos 3,7 pJ, mientras que leer 32 bits de DRAM cuesta unos 640 pJ: mover el dato cuesta unas $640/3{,}7 \approx 170$ veces más que operar con él. Las técnicas de los temas 1 a 3 (matrices sistólicas, *tiling*, fusión, FlashAttention) ahorran energía por la misma razón por la que ahorran tiempo: reducen el tráfico con la memoria lejana.

6.4 El límite de Landauer

Teorema 6.5 (Principio de Landauer). *Borrar un bit de información, es decir, llevar un sistema físico que almacena un bit desconocido (0 o 1 con igual probabilidad) a un estado fijo, en contacto con un entorno a temperatura T, disipa en ese entorno un calor*

$$Q \;\geq\; k_{\mathrm{B}} T \ln 2,$$

donde $k_{\mathrm{B}} = 1{,}380649 \cdot 10^{-23}$ J/K es la constante de Boltzmann.

Observación 6.7. El principio solo limita las operaciones *irreversibles*, que destruyen información. Una suma $c = a + b$ que sobrescribe a y b es irreversible; la computación reversible, que conserva la información necesaria para deshacer cada paso, podría en principio operar por debajo de $k_{\mathrm{B}} T \ln 2$ por operación. El vínculo con el módulo 6 es directo: $k_{\mathrm{B}} \ln 2$ es la entropía de un bit expresada en julios por kelvin.

Bibliografía

[1] J. Austin, S. Douglas, R. Frostig, A. Levskaya, C. Chen, S. Vikram, F. Lebron, P. Choy, V. Ramasesh, A. Webson y R. Pope. *How to Scale Your Model*. Google DeepMind, 2025. Parte 1, capítulo «All About Rooflines». https://jax-ml.github.io/scaling-book/

[2] T. Chen, B. Xu, C. Zhang y C. Guestrin. Training deep nets with sublinear memory cost. Prepublicación, 2016. https://arxiv.org/abs/1604.06174

[3] A. Chowdhery, S. Narang, J. Devlin *et al.* PaLM: Scaling language modeling with Pathways. Prepublicación, 2022. Publicado en *Journal of Machine Learning Research*, 24, 2023. https://arxiv.org/abs/2204.02311

[4] Carnegie Mellon University. 15-418/15-618: Parallel Computer Architecture and Programming. Primavera de 2026. Clases «Modern multicore processors», «Locality, communication, and contention», «Graphic processing units and CUDA» e «Interconnection networks». https://www.cs.cmu.edu/afs/cs/academic/class/15418-s26/www/schedule.html

[5] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 1, «Basics», versión 26.0.3. Stanford University, primavera de 2026. §3.5 (*Resource accounting*) y §4.3 (AdamW). https://github.com/stanford-cs336/assignment1-basics

[6] Equipo docente de CS336. CS336: Language Modeling from Scratch. Tarea 2, «Systems», versión 26.1.3. Stanford University, primavera de 2026. https://github.com/stanford-cs336/assignment2-systems

[7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra y C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2205.14135

[8] S. Han. 6.5940: TinyML and Efficient Deep Learning Computing. Massachusetts Institute of Technology, otoño de 2024. Clases 5 y 6 («Quantization») y 19 y 20 («Distributed Training»). https://hanlab.mit.edu/courses/2024-fall-65940

[9] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Clases sobre GPU y TPU, clase 6 (*kernels* y Triton) y clase 7 («Parallelism»). https://github.com/stanford-cs336/lectures

[10] J. L. Hennessy y D. A. Patterson. *Computer Architecture: A Quantitative Approach*, 6.ª ed. Morgan Kaufmann, 2019. Cap. 4, «Data-Level Parallelism in Vector, SIMD, and GPU Architectures»; cap. 6, «The Warehouse-Scale Computer»; cap. 7, «Domain-Specific Architectures».

[11] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2.ª ed. SIAM, 2002. Caps. 2 a 4.

[12] J. Hoffmann, S. Borgeaud, A. Mensch *et al.* Training compute-optimal large language models. Prepublicación, 2022. https://arxiv.org/abs/2203.15556

[13] J.-W. Hong y H. T. Kung. I/O complexity: The red-blue pebble game. En *Proceedings of the 13th Annual ACM Symposium on Theory of Computing (STOC)*, 1981.

[14] M. Horowitz. Computing's energy problem (and what we can do about it). En *IEEE International Solid-State Circuits Conference (ISSCC)*, págs. 10–14, 2014.

[15] Y. Huang, Y. Cheng, A. Bapna, O. Firat, M. X. Chen, D. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu y Z. Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. En *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. https://arxiv.org/abs/1811.06965

[16] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.* In-datacenter performance analysis of a tensor processing unit. En *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017. https://arxiv.org/abs/1704.04760

[17] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu y D. Amodei. Scaling laws for neural language models. Prepublicación, 2020. https://arxiv.org/abs/2001.08361

[18] R. Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.

[19] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh y H. Wu. Mixed precision training. En *International Conference on Learning Representations (ICLR)*, 2018. https://arxiv.org/abs/1710.03740

[20] P. Micikevicius, D. Stosic, N. Burgess *et al.* FP8 formats for deep learning. Prepublicación, 2022. https://arxiv.org/abs/2209.05433

[21] M. Milakov y N. Gimelshein. Online normalizer calculation for softmax. Prepublicación, 2018. https://arxiv.org/abs/1805.02867

[22] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee y M. Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. En *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021. https://arxiv.org/abs/2104.04473

[23] NVIDIA. NVIDIA A100 Tensor Core GPU, hoja de especificaciones; y *Tuning CUDA Applications for NVIDIA Ampere GPU Architecture*. https://www.nvidia.com/en-us/data-center/a100/

[24] P. Patarasuk y X. Yuan. Bandwidth optimal all-reduce algorithms for clusters of workstations. *Journal of Parallel and Distributed Computing*, 69(2), 2009.

[25] S. Rajbhandari, J. Rasley, O. Ruwase e Y. He. ZeRO: Memory optimizations toward training trillion parameter models. En *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20)*, 2020. https://arxiv.org/abs/1910.02054

[26] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper y B. Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. Prepublicación, 2019. https://arxiv.org/abs/1909.08053

[27] S. Williams, A. Waterman y D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.